ЛАБОРАТОРНА РОБОТА 5

Серіалізація об’єктів .Net

Мета лабораторної роботи – дослідити механізм серіалізації об’єктів .Net

Теоретичні відомості

Серіалізація представляє процес перетворення будь-якого об'єкта в потік байтів. Після перетворення можна цей потік байтів або записати на диск, або зберегти його тимчасово в пам'яті. А при необхідності можна виконати зворотний процес - десеріалізацію, тобто отримати з потоку байтів раніше збережений об'єкт.

Щоб об'єкт певного класу можна було серіалізовать, треба цей клас помітити атрибутом Serializable:

```
[Serializable]
public class Person
{
    public string Name { get; set; }
    public string SecondName { get; set; }
    public string SurName { get; set; }
}
```

При відсутності даного атрибута об'єкт Person не зможе бути серіалізованим, і при спробі серіалізації буде викинуто виключення **SerializationException**.

Якщо якийсь член помічаний атрибутом **NonSerialized**, то він не буде серіалізований.

```
[NonSerialized]
public string AccNumber { get; set; }
```

В .NET можна використовувати такі формати:

- бінарний
- SOAP (Simple Object Access **Protocol)**
- xml
- JSON

Для кожного формату передбачений свій клас:

- для серіалізації в бінарний формат - клас **BinaryFormatter**,
- для формату SOAP - клас **SoapFormatter,**
- для xml - **XmlSerializer**,
- для json - **DataContractJsonSerializer**.

Серіалізація в XML. XmlSerializer

XmlSerializer передбачає деякі обмеження. Наприклад, клас, який підлягає серіалізації, повинен мати стандартний конструктор без параметрів. Також серіалізації підлягають тільки відкриті члени. Якщо в класі є поля або властивості з модифікатором private, то при серіалізації вони будуть ігноруватися.

```
[Serializable]
public class Person
{
    public string Name { get; set; }
    public string SecondName { get; set; }
    public string SurName { get; set; }
}
```



Фрагмент кода програми:

```
public partial class Form1 : Form
{
    Person person;
    XmlSerializer formatter;
    FileStream fs;
    public Form1()
    {
        InitializeComponent();
    }

// метод-обробник події натискання кнопки "Зберегти параметри"
    private void button1_Click(object sender, EventArgs e)
    {
        // об'єкт для серіалізації
        person = new Person();
        person.Name = textBox1.Text;
        person.SecondName = textBox2.Text;
        person.SurName = textBox3.Text;
        // передача в конструктор тип класу
        formatter = new XmlSerializer(typeof(Person));
        // отримуємо потік, куди будемо записувати серіалізовані об'єкт
        fs = new FileStream("persons.xml", FileMode.OpenOrCreate);
        formatter.Serialize(fs, person);
        fs.Close();

    }
```

Після натискання кнопки «Зберегти параметри» буде створений файл persons.xml. Якщо ми відкриємо файл persons.xml, то побачимо зміст нашого об'єкта:

```xml
<?xml version="1.0"?>
<Person xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <Name>111</Name>
  <SecondName>222</SecondName>
  <SurName>333</SurName>
</Person>
```

Десеріалізація

Після натискання кнопки «Встановити параметри» виконується метод:

```
private void button2_Click(object sender, EventArgs e)
{
    // десеріалізація
    Person newPerson;
    if (File.Exists("persons.xml"))
    {
        fs = new FileStream("persons.xml", FileMode.Open);
        formatter = new XmlSerializer(typeof(Person));
        newPerson = (Person)formatter.Deserialize(fs);
        fs.Close();
    }
    else
        newPerson = new Person();
    textBox1.Text = newPerson.Name;
    textBox2.Text = newPerson.SecondName;
    textBox3.Text = newPerson.SurName;
    fs.Close();
}
```

Після натискання кнопки «Встановити параметри» будуть встановлені параметри:





Бінарна серіалізація. BinaryFormatter

Клас BinaryFormatter визначено в просторі імен **System.Runtime.Serialization.Formatters.Binary**, то на самому початку підключаємо його.

```
using System.IO;
using System.Xml.Serialization;
using System.Runtime.Serialization.Formatters.Binary;
```

```
public partial class Form1 : Form
{
    Person person;
    BinaryFormatter formatter;
    FileStream fs;
    public Form1()
    {
        InitializeComponent();
    }

// метод-обробник події натискання кнопки "Зберегти параметри"
        private void button1_Click(object sender, EventArgs e)
        {
            // об'єкт для серіалізації
            person = new Person();
            person.Name = textBox1.Text;
            person.SecondName = textBox2.Text;
            person.SurName = textBox3.Text;

            formatter = new BinaryFormatter();
            // отримуємо потік, куди будемо записувати серіалізований об'єкт
            fs = new FileStream("persons.dat", FileMode.OpenOrCreate);
            formatter.Serialize(fs, person);
            fs.Close();

        }
```

Після натискання кнопки «Зберегти параметри» буде створений бінарний файл persons.dat.

Десеріалізація

Після натискання кнопки «Встановити параметри» виконується метод:

```
private void button2_Click(object sender, EventArgs e)
{
    // десеріалізація
    Person newPerson;
    if (File.Exists("persons.dat"))
    {
        fs = new FileStream("persons.dat", FileMode.Open);
        formatter = new BinaryFormatter();
        newPerson = (Person)formatter.Deserialize(fs);
        fs.Close();
    }
    else
        newPerson = new Person();
    textBox1.Text = newPerson.Name;
    textBox2.Text = newPerson.SecondName;
    textBox3.Text = newPerson.SurName;
    fs.Close();
}
```

Після натискання кнопки «Встановити параметри» будуть встановлені параметри.

Рефлексія

Рефлексія (Reflection) – надання коду, що виконується, інформації про нього самого. Рефлексія є процес аналізу типів (структури типів) в ході виконання програми (збірки).

В .NET рефлексія реалізується властивостями і методами класу **System.Type** і класів простору імен **System.Reflection**. Клас System.Type містить методи, що дозволяють отримувати інформацію про типи додатків.

Використання членів класу Type дозволяє отримати інформацію про

типи (GetType (string)), конструктори (GetConstructors ()), методи (GetMethods ()),

дані-члени (GetFields ()), властивості (GetProperties ()), події, оголошені в класі (GetEvents ()), модуль, збірка, в якій реалізується даний клас.

Операція typeof(), застосовується для побудови об'єкта-представника класу System.Type. Вираз, побудований на основі операції typeof, має такий вигляд:

typeof(type)

Приклади застосування:

```
using System;
using System.Reflection;

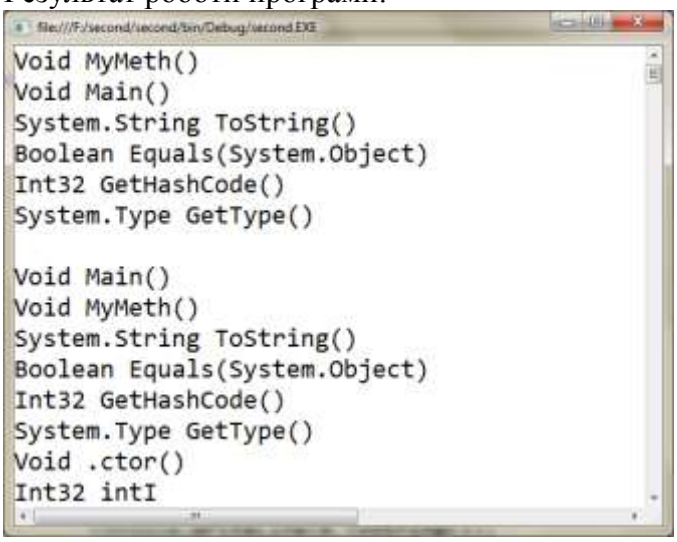
public class MyClass
{
    public int intI;
    static public void MyMeth()
    {
    }

    public static void Main()
    {
        Type t = typeof(MyClass);


        MethodInfo[] x = t.GetMethods();
        foreach (MethodInfo m in x)
        {
            Console.WriteLine(m.ToString());
        }

        Console.WriteLine();
        MemberInfo[] x2 = t.GetMembers();
        foreach (MemberInfo m in x2)
        {
            Console.WriteLine(m.ToString());
        }
        Console.ReadKey();
    }
}
```

Результат роботи програми:



Завдання

1. Створити об’єкт певного класу відповідно варіанту завдання. Ініціалізувати поля і властивості об’єкта. Зберегти стан об’єкта, використовуючи механізм бінарної і XML-серіалізації.

2. Після перезапуску програми встановити параметри об’єкта, використовуючи механізм бінарної і XML-десеріалізації.

3. Отримати інформацію про власивості і методи класу, використовуючи механізм рефлексії.

Варіанти:

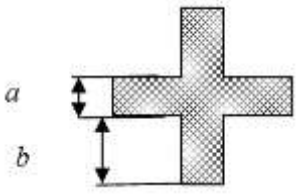
1. Геометрічна фігура «Смайлік»: його екземпляр містить розмір радіуса особи R і радіуса r очей, його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням R і r.
   Метод Draw() рисує у вікні форми зображення фігури.



2. Геометрічна фігура «хрест», такий, що:

a) його екземпляр містить розмір два розміри a і b.

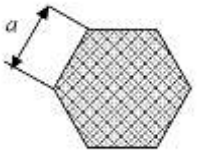
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b.
   Метод Draw() рисує у вікні форми зображення фігури.



3. Геометрічна фігура «Шайба» правильний шестикутник, такий, що.

   a) його екземпляр містить розмір сторони a.

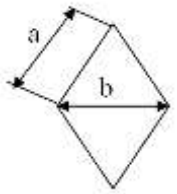
   b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a.
   Метод Draw() рисує у вікні форми зображення фігури.



4. Геометрічна фігура «Ромб»:

a) його екземпляр містить розміри a і b.

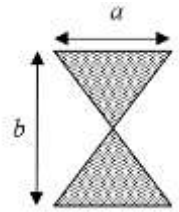
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b.
Метод Draw() рисує у вікні форми зображення фігури.



5. Геометрічна фігура "пісочний годинник" :

a) його екземпляр містить розміри a і b.

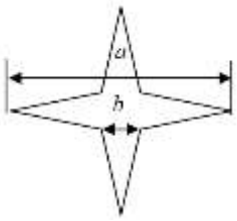
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b.
Метод Draw() рисує у вікні форми зображення фігури.



6. Геометрічна фігура «чотирикутний зірка»:

a) його екземпляр містить розміри a і b.

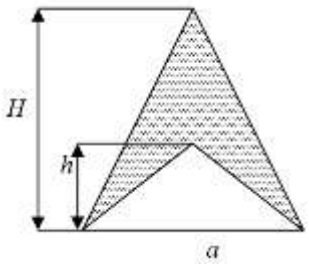
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b.
Метод Draw() рисує у вікні форми зображення фігури.



7. Геометрічна фігура «Вігвам» (Заштрихована фігура):

a) його екземпляр містить розміри a, H і h.

b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням.
Метод Draw() рисує у вікні форми зображення фігури.



8. Геометрічна фігура «трапеція»

a) його екземпляр містить розмір двох основ і висоти.

b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням основ і висоти.

a) метод Draw() рисує у вікні форми зображення фігури.

Варіант 11. Створити клас «буква», такий, що:

a) він містить тип Color, що позначає кольор букви, Font шрифт.
b) його примірник містить код букви латинського алфавіту і колір цієї літери.
c) його конструктор без параметра створює екземпляр зі значенням «пробіл», колір білий, а конструктор з параметрами створює екземпляр з відповідними значеннями букви. Для вводу параметрів можна використовувати діалогові вікна ColorDialog I FontDialog.
d) метод Draw() рисує у вікні форми зображення фігури.

Варіант 12. Створити клас «правильний багатокутник», такий, що:

a) його примірник містить кількість вершин (не менше 3) і радіус описаного кола (не менше 1.0).
b) його конструктор без параметра створює трикутник з мінімальним значенням радіусу, а конструктор з параметрами створює екземпляр з відповідною кількістю вершин і радіусом.
c) метод Draw() рисує у вікні форми зображення фігури.

Варіант 13. Створити базовий клас «смайлик», такий, що:

a) його примірник містить розмір радіусу особи R і радіуса r очей.
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням R і r.
c) метод Draw() рисує у вікні форми зображення фігури.



Варіант 14. Створити базовий клас «правильний п'ятикутник» такий, що:

a) його примірник містить радіус r описаного кола (не менше 1.0).
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням r.
c) метод Draw() рисує у вікні форми зображення фігури.

Варіант 10. Створити клас «вектор», такий, що:

a) його примірник містить три координати - змінні з плаваючою точкою. Особливість значень координат: знак третьої координати автоматично стає таким, що їх твір завжди позитивно.
b) його конструктор без параметра створює екземпляр зі значенням 0.0, 0.0, 0.0, а конструктор з параметрами створює екземпляр з відповідними значеннями координат.
c) метод Draw() рисує у вікні форми зображення фігури.
d) Геометрічна фігура «еліпс»
e) Геометрічна фігура «правильний п'ятикутник»